케이녹 웹해킹 1회차 과제

17기 최동현 (202511688)

1. HTTP/HTTPS (HyperText Transfer Protocol)

- **HTTP/HTTPS란?**

1) HTTP(*HyperText Transfer Protocol*)는 (*클라이언트*)와 (*서버*) 간 HTML 문서, 이미지 등의 데이터를 주고받기 위한 **OSI 7 계층** 중 **Application Layer(*L7*)**의 통신 프로토콜입니다.
기본 포트로 **Port 80**번을 사용하며, 모든 데이터가 평문(Plain Text) 형태로 전송됩니다.

2) HTTPS(*HyperText Transfer Protocol Secure*)는 HTTP 하부에 **SSL/TLS(*Secure Sockets Layer/Transport Layer Security*)** 프로토콜을 추가하여 데이터 **기밀성**, **무결성**, **인증**을 보장할 수 있도록 하는 HTTP 통신의 **보안 규약**입니다.
기본 포트로 **Port 443**번을 사용하며, **공개키 기반 구조**(**P**ublic **K**ey **I**nfrastructure)를 사용하여 **페이로드**(**Payload**)를 암호화하여 전송합니다.

1. HTTP/HTTPS (HyperText Transfer Protocol)

- HTTP/HTTPS란? (HTTP)

```
100%  10%  13 GB  0.0 kB↓  0.0 kB↑
~/Desktop/CHOI  curl -v -s http://www.example.org 1>/dev/null  ✔  19:16:52
* Host www.example.org:80 was resolved.
* IPv6: (none)
* IPv4: 172.66.157.237, 104.20.26.136
*   Trying 172.66.157.237:80...
* Connected to www.example.org (172.66.157.237) port 80
> GET / HTTP/1.1
> Host: www.example.org
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Wed, 29 Apr 2026 10:16:53 GMT
< Content-Type: text/html
```

요청, 호스트, 사용툴(브라우저)를
확인할 수 있다.

1. HTTP/HTTPS (HyperText Transfer Protocol)

- HTTP/HTTPS란? (HTTPS)

```
~/Desktop/CHOI  curl -v -s https://www.example.org 1>/dev/null  ✔ 19:10:31
* Host www.example.org:443 was resolved.
* IPv6: (none)
* IPv4: 104.20.26.136, 172.66.157.237
*   Trying 104.20.26.136:443...
* Connected to www.example.org (104.20.26.136) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
} [320 bytes data]
*  CAfile: /etc/ssl/cert.pem
*  CApath: none
* (304) (IN), TLS handshake, Server hello (2):
{ [122 bytes data]
* (304) (IN), TLS handshake, Unknown (8):
{ [19 bytes data]
* (304) (IN), TLS handshake, Certificate (11):
{ [3669 bytes data]
* (304) (IN), TLS handshake, CERT verify (15):
{ [79 bytes data]
* (304) (IN), TLS handshake, Finished (20):
{ [52 bytes data]
* (304) (OUT), TLS handshake, Finished (20):
} [52 bytes data]
* SSL connection using TLSv1.3 / AEAD-AES256-GCM-SHA384 / [blank] / UNDEF
* ALPN: server accepted h2
```

실제 TLS Handshake가 이뤄지는 모습

2. 데이터 전송 방식 (GET, POST)

- **데이터 전송 방식이란?**

: HTTP 통신 규약은 서버에 요청을 보낼 때, 목적에 따라 다른 두 메서드(Method)를 사용합니다.

1) GET Method : 서버로부터 특정 **리소스(데이터)**를 **조회**할 때 주로 사용됩니다.
전송하려는 **데이터(파라미터)**를 URI(Uniform Resource Identifier)의 끝에 쿼리 스트링(Query String) **<** /index.php**?list=news >** 형태로 결합하여 전송합니다.

2) POST Method : 서버의 상태를 변경하거나 새로운 데이터를 **생성/제출(Create/Update)**할 때 사용합니다.
전송하려는 데이터를 **HTTP Request**의 본문인 Message Body에 포함하여 전송하며, 서버가 메시지 바디를 올바르게 해석하도록 HTTP 헤더에 Content-Type과 Content-Length를 명시해야 합니다.

2-1. 데이터 전송 방식 (GET, POST)

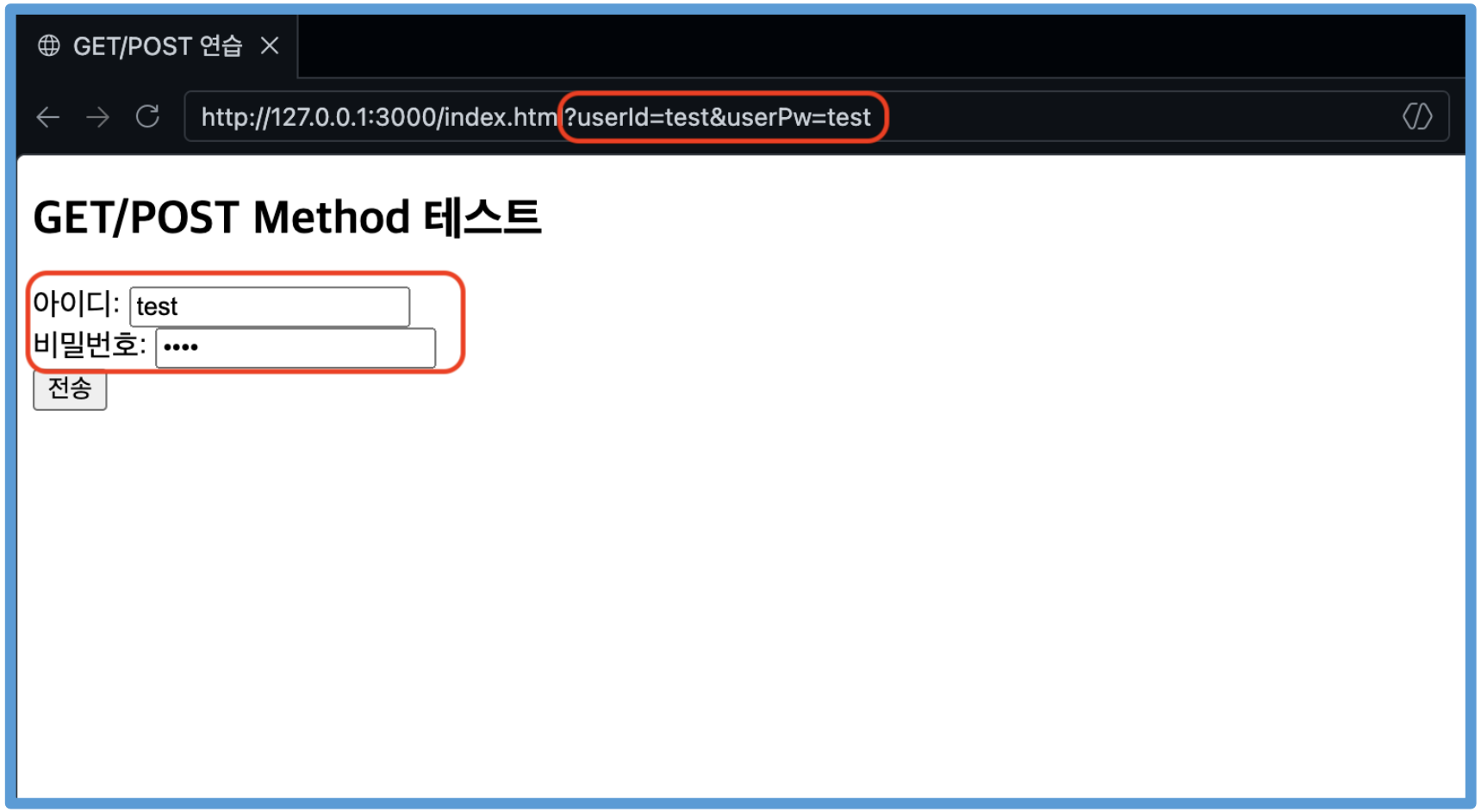
- 데이터 전송 방식이란? (헤더 확인)

```
100%  8%  13 GB  1.0 kB↓  0.0 kB↑
~/Desktop/CHOI  curl --head https://www.example.org  ✔ 19:01:11
HTTP/2 200   200 OK
date: Wed, 29 Apr 2026 10:01:20 GMT
content-type: text/html
server: cloudflare
last-modified: Sat, 18 Apr 2026 00:51:00 GMT
allow: GET, HEAD   GET Method
accept-ranges: bytes
age: 13859
cache-control: max-age=14400
cf-cache-status: HIT
cf-ray: 9f3d77007f73fcd4-FUK

~/Desktop/CHOI  ✔ 19:01:20
```

2-2. 데이터 전송 방식 (GET, POST)

- 데이터 전송 방식이란? (GET Method)



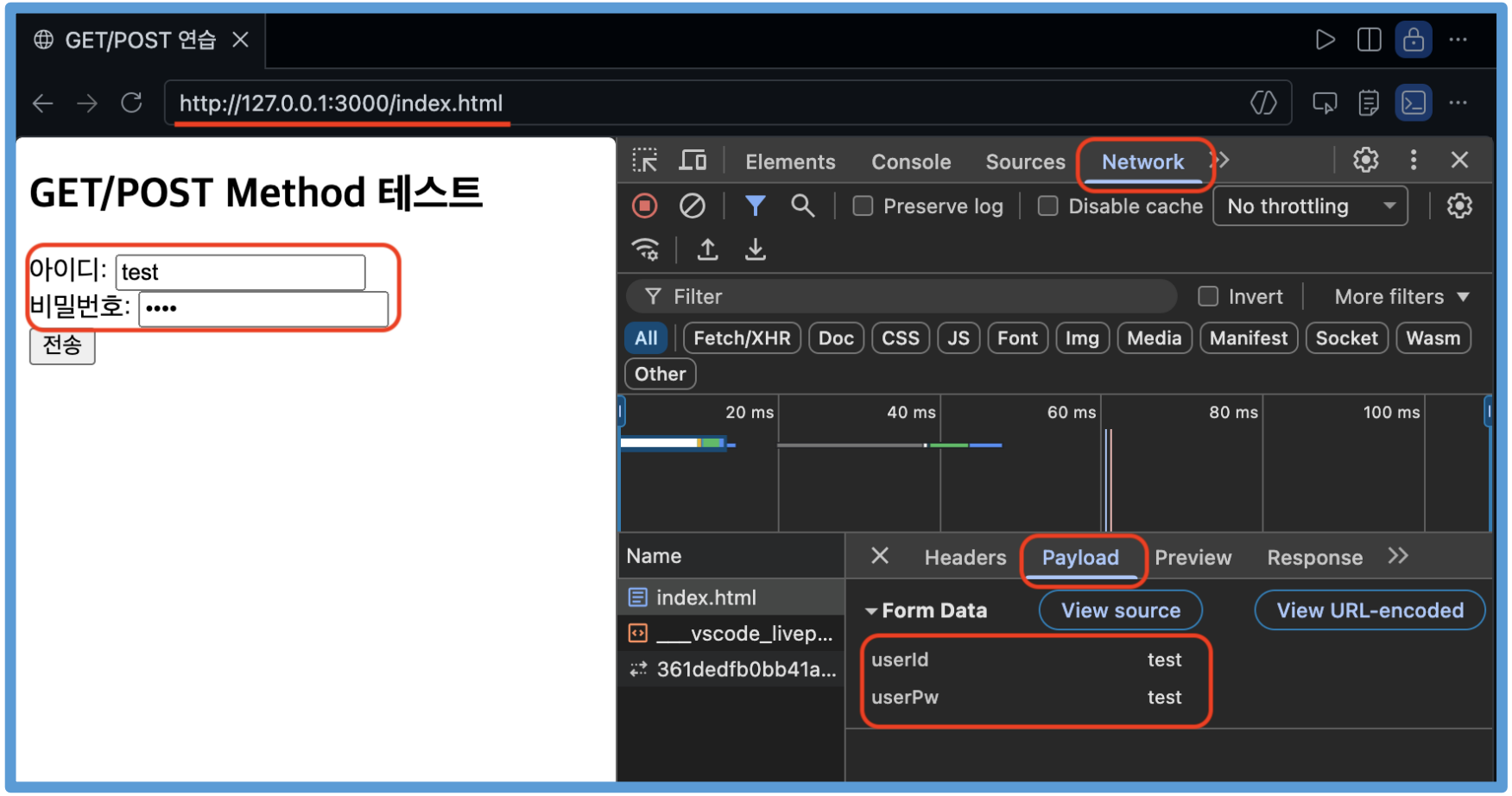
2-3. 데이터 전송 방식 (GET, POST)

- 데이터 전송 방식이란? (POST Method)

```
100%  10%  13 GB  0.0 kB↓  0.0 kB↑
~/Desktop/CHOI  curl -v -X POST -d "userId=test&userPw=test" http://127.0.0.1:300
0
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 127.0.0.1:3000...
* Connected to 127.0.0.1 (127.0.0.1) port 3000
> POST / HTTP/1.1
> Host: 127.0.0.1:3000
> User-Agent: curl/8.7.1
> Accept: */*
> Content-Length: 23
> Content-Type: application/x-www-form-urlencoded
>
* upload completely sent off: 23 bytes
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=UTF-8
< Content-Length: 518
< Cross-Origin-Resource-Policy: cross-origin
```

2-4. 데이터 전송 방식 (GET, POST)

- 데이터 전송 방식이란? (POST Method)



2. 데이터 전송 방식 (GET, POST)

- **GET/POST Method (보안 취약점)**

1) GET Method 취약점 : GET 방식으로 민감한 정보(비밀번호 or 개인정보)를 전송하면 해당 데이터가 **URI**에 명시적으로 노출되기 때문에 GET은 정보 **조회 요청** 시에만 사용하는 것이 좋다.

2) POST Method 취약점 : POST 방식은 URI에 데이터가 노출되지 않아 **안전해 보이지만 . . .** 암호화가 적용되지 않은 상태라면 네트워크 상에서 조작이 가능합니다. 해커는 인터셉트 프록시 툴을 이용해 HTTP Request Body를 중간에 낚아채 제품 결제 시 price=10000 와 같은 파라미터를 price=0으로 변조하여 서버로 전송하는 등.

암호화 되지 않은 **POST 파라미터 변조 공격(Parameter Tampering)**을 수행하기도 합니다.

3. 프록시 (Proxy)

- **프록시 (Proxy)란?**

  : **프록시(Proxy)**는 **Client**와 **Endpoint Server** 사이, 통신을 중계하는 **대리 서버(Intermediary Server)** 시스템입니다. (IT 환경의 경우 Firewall 이라고 볼 수 있음)

사진 출처 : Jung's (https://jung-story.tistory.com/117)

3-1. 프록시 (Proxy)

• 포워드 프록시 (Forward Proxy)란?

: **포워드 프록시(Forward Proxy)**는 클라이언트(사용자)가 위치한 **내부 네트워크** 혹은 **클라이언트 측**에 구성되는 **정방향 중계 서버**입니다.

클라이언트가 외부 인터넷(웹 서버)으로 직접 요청을 보내는 대신, **Forward Proxy**로 요청을 전송하면 프록시 서버가 이를 대신 받아 외부 서버로 **전달(Forwarding)**하고 그 응답을 클라이언트에게 반환합니다.

(**장점** : *IP 노출 방지 및 특정 사이트 접속 제한 가능!*) -> *마치 네트워크 방화벽과 비슷함*

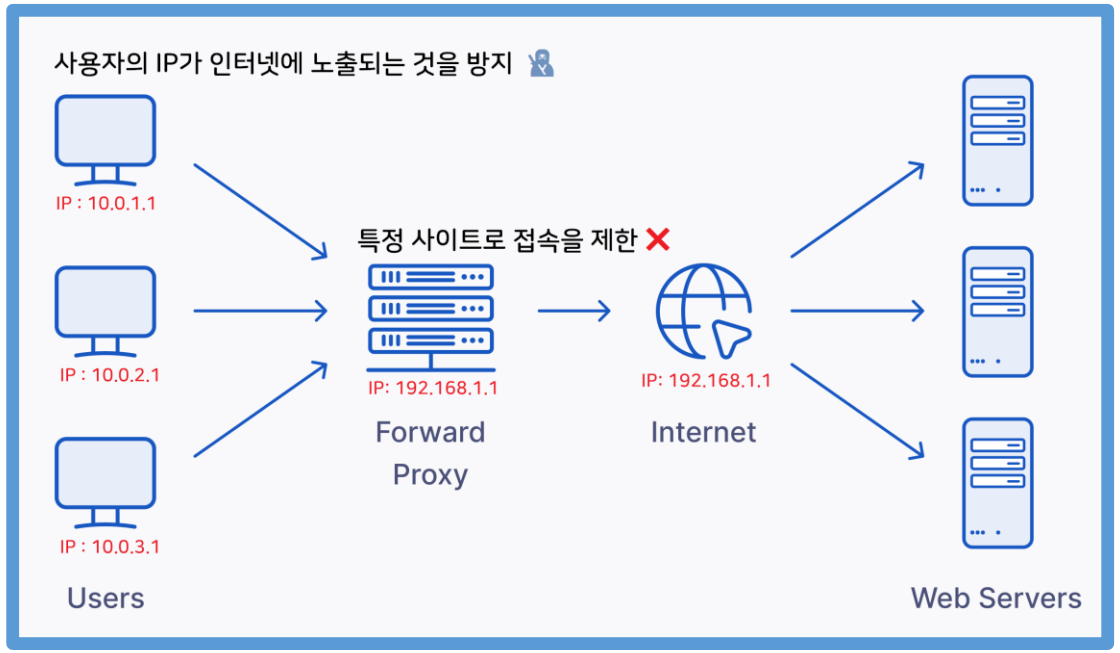


사진 출처 : vinca (https://velog.io/@rockwellvinca/Reverse-Proxy와-Forward-Proxy-쉽게-이해하기

3-2. 프록시 (Proxy)

• 리버스 프록시 (Reverse Proxy)란?

: **리버스 (Reverse Proxy)**는 외부 **인터넷(클라이언트)**과 **내부 웹 서버(Backend Server)**들 사이에 위치하여, 웹 서버를 보호하고 트래픽을 분산하는 **역방향 중계 서버**입니다.

사용자는 내부 웹 서버의 존재나 실제 IP 주소를 알 수 없으며, 오직 Reverse Proxy의 공개된 IP로만 요청을 보냅니다. 프록시 서버가 요청을 분석하여 적절한 내부 서버(WAS, DB 등)로 트래픽을 전달합니다.

(**장점** : *로드밸런싱->라운드로빈(내부 서버 부하분산), 암호화(SSL/TLS) 적용 하는 등 퍼포먼스 가능!*)

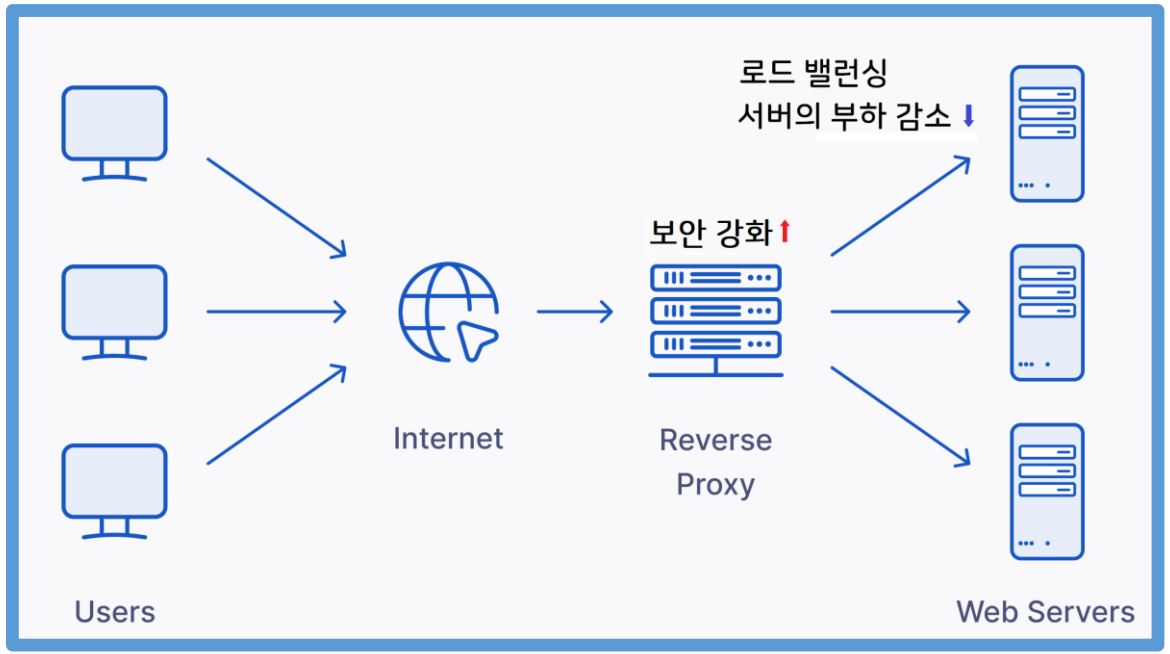


사진 출처 : vinca (https://velog.io/@rockwellvinca/Reverse-Proxy와-Forward-Proxy-쉽게-이해하기

3-3. 프록시 (Proxy)

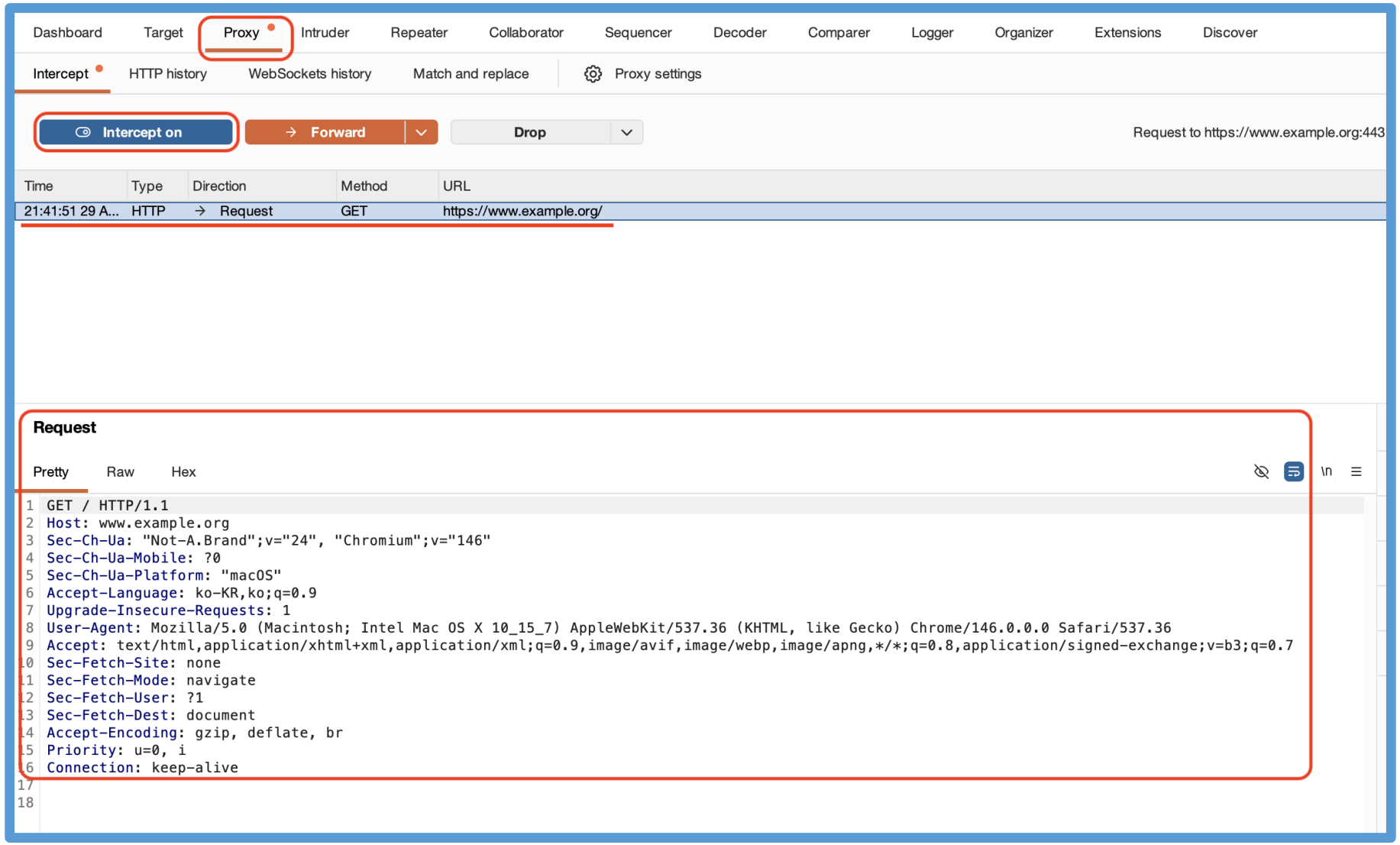
- **인터셉트 프록시 (Intercept Proxy)란?**

: **인터셉트 프록시 (Intercept Proxy)**는 일반적인 프록시의 전달 기능에 일시 정지 및 변조 기능을 추가한 보안 점검용 전문 도구입니다. (대표적인 툴로 **Burp Suite**가 있습니다.)

이 프록시는 클라이언트와 서버 사이의 통신 경로에 개입하여 브라우저가 생성한 HTTP Request 패킷이 네트워크 카드(NIC)를 타고 인터넷으로 나가기 직전에 메모리 상에 트래픽을 가둬두는 역할을 합니다.

이 단계에서 **패킷**의 평문(Plain Text) 구조를 직접 **분석** 또는 서버가 예상치 못한 악의적인 페이로드를 바이트 단위로 수정하여 **전송(Forward)**할 수 있습니다. 응답(Response) 패킷 또한 브라우저에 도달하기 전 가로채어 클라이언트 로직을 조작할 수도 있습니다.

4. Burp Suite 기본 사용법 (프록시 인터셉트)



감사합니다

17기 최동현